

PRINCETON UNIVERSITY / RESEARCH TALK

Linear Attention, State Spaces, and RLT

Memory updates and computation across tokens

Yifan Zhang Princeton ECE / October 2026

Based on *A Note on Linear Attention and State Space Models*
and *Recurrent Looped Transformer*

The state-space view

$$h_t = \mathcal{F}_\theta(h_{t-1}; u_t), \quad y_t = \mathcal{G}_\theta(h_t; u_t)$$

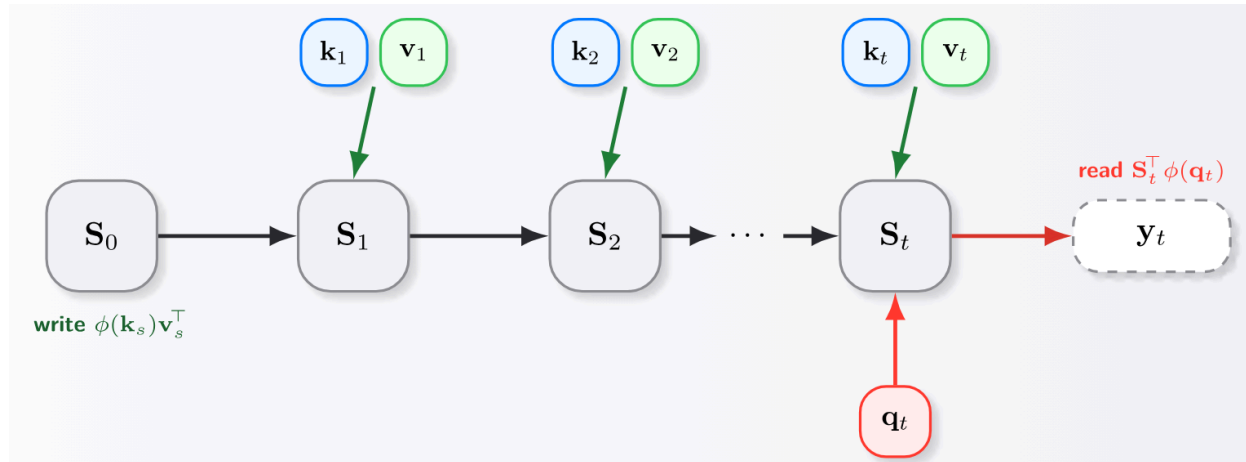
Object	Meaning	Question to ask
State h_t	Information carried into the next update	What must persist?
Input u_t	New data and conditioning features	What is available before the update?
Transition $\mathcal{F}_\theta$	The rule that transforms the state	Affine or nonlinear in the old state?

The recurrence index can represent token position or network depth.

Note: Yifan Zhang and Sun-Yuan Kung

RLT: Yifan Zhang, Jichen Feng, Shihan Qin, Sun-Yuan Kung, and Mengdi Wang

Linear attention as recurrent memory

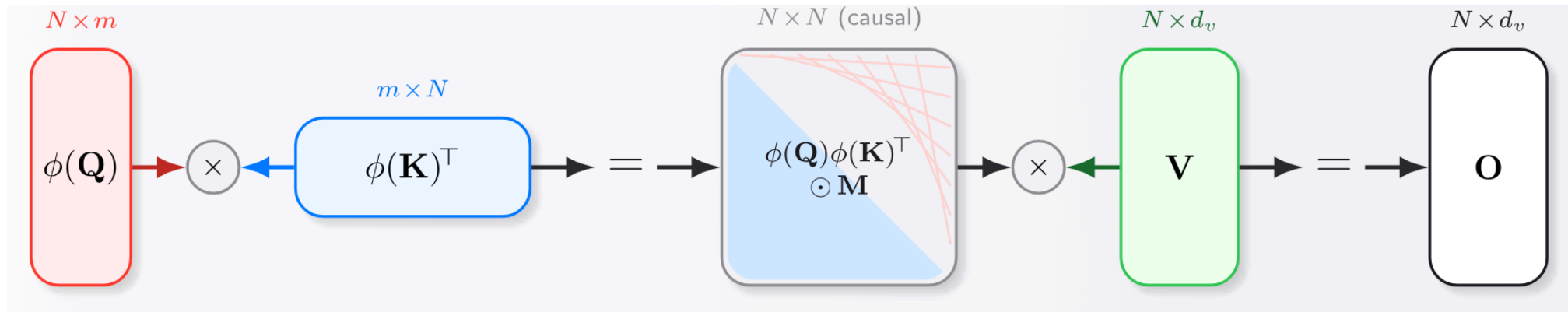


Key features $a_t = \phi(k_t)$, query features $b_t = \phi(q_t)$. Read after writing the current token.

$$S_t = S_{t-1} + a_t v_t^\top, \quad z_t = z_{t-1} + a_t, \quad y_t = \frac{S_t^\top b_t}{z_t^\top b_t + \varepsilon}$$

State size $O(md_v)$ stays fixed as the sequence grows. The vector z_t supplies the normalizer.

The same computation in attention form



$$S_t = \sum_{j \leq t} a_j v_j^\top \quad \Rightarrow \quad S_t^\top b_t = \sum_{j \leq t} (b_t^\top a_j) v_j$$

$$\mathbf{Y} = (\mathbf{M} \odot \Phi_Q \Phi_K^\top) \mathbf{V}, \quad M_{tj} = \mathbf{1}\{j \leq t\}$$

Same outputs, different evaluation: a running state or a causal $T \times T$ score matrix.

Unnormalized outputs shown. Normalized attention also divides each row by its kernel sum.

Note / Original recurrent and parallel views / Katharopoulos et al., 2020

Selective SSMs and state-space duality

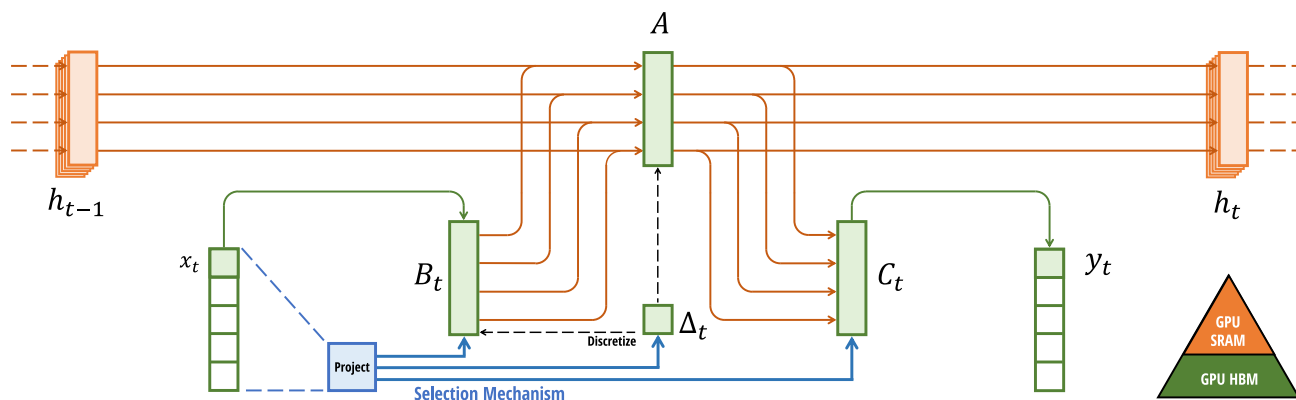
Selective State Space Model
with Hardware-aware State Expansion

The SSM core

$$h_t = \bar{A}_t h_{t-1} + \bar{B}_t x_t$$

$$y_t = C_t^\top h_t$$

Mamba selects Δ_t, B_t, C_t from layer inputs.
 $\bar{A}_t = \exp(\Delta_t A)$.



Scalar-transition SSD, with zero initial state

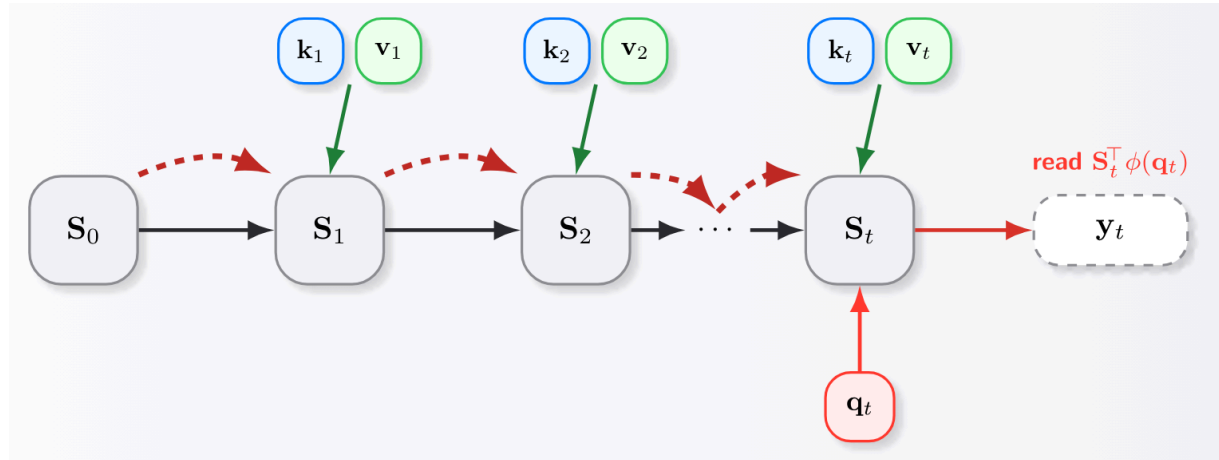
$$S_t = \rho_t S_{t-1} + B_t x_t^\top, \quad y_t = S_t^\top C_t$$

$$Y = (L \odot C B^\top) X, \quad L_{tj} = \mathbf{1}\{j \leq t\} \prod_{r=j+1}^t \rho_r$$

C : queries, B : keys, X : values. Setting $\rho_t = 1$ recovers causal linear attention.

Note / Selective SSMs and SSD / Gu and Dao / Mamba, Fig. 1 / Dao and Gu / SSD

DeltaNet writes the retrieval error



$$\begin{aligned}
 S_t &= S_{t-1} + \eta_t a_t (v_t - S_{t-1}^\top a_t)^\top \\
 &= (I - \eta_t a_t a_t^\top) S_{t-1} + \eta_t a_t v_t^\top
 \end{aligned}$$

Write the retrieval error. The update is affine in the old state when the inputs are fixed.
 Green arrows: writes. Dashed red arrows: key-dependent edits to the old memory.

Note / Delta Networks and original figure / Yang et al., 2024

Why affine updates admit a scan

Fix a layer's input sequence. Represent one transition by (A_t, c_t) :

$$h_t = A_t h_{t-1} + c_t$$

$$\underbrace{(A_2, c_2) \circ (A_1, c_1)}_{\text{two consecutive updates}} = (A_2 A_1, A_2 c_1 + c_2)$$

Closed under composition

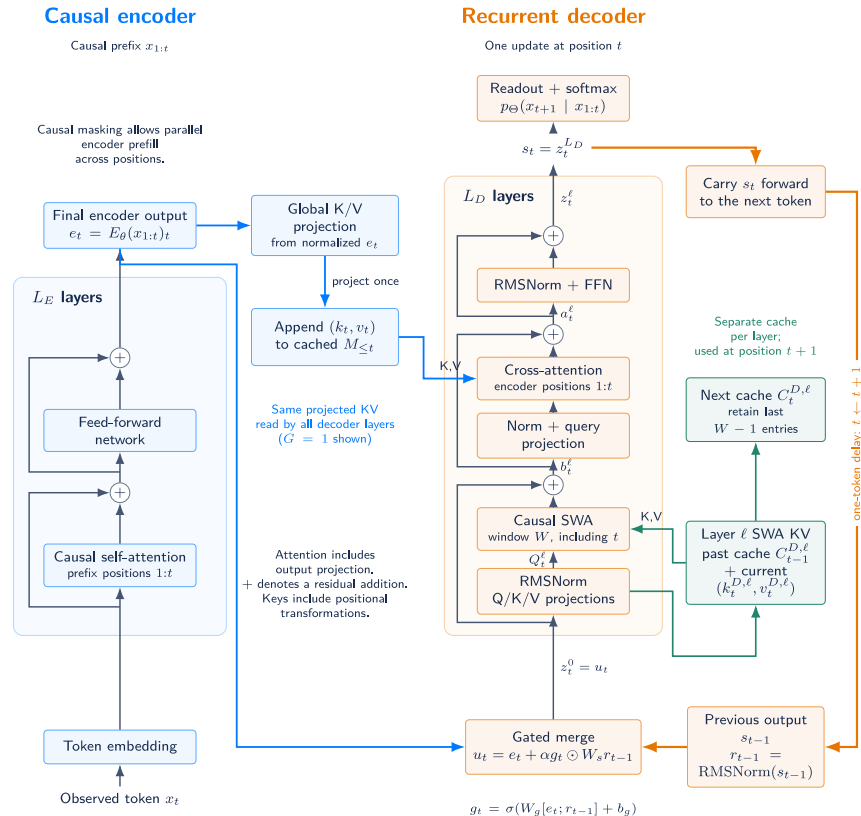
A segment of updates is still one affine map.
Prefix composition is associative.

Structure controls cost

Additive, diagonal, scalar, and low-rank forms
support efficient implementations.

A general nonlinear recurrence has no comparable fixed-size affine summary.

RLT: a Transformer decoder as the recurrent cell



Causal encoder

Known tokens produce e_t and prefix memory $M_{\leq t}$.

Gated feedback

Merge e_t with the previous final decoder output s_{t-1} .

Full decoder update

SWA, cross-attention, and an FFN transform the merged input.

SWA = sliding-window attention.

Every output predicts the next token.

RLT's explicit state-transition function

Complete decoder state: $H = (s, C^D)$, including the feedback vector and local KV cache.

$$\mathcal{F}_{\Theta}(H; e, M, t) := D_{\phi}(\text{Merge}(e, s); M, C^D, t)$$

$$\begin{aligned} r &= \text{Norm}_s(s), & g &= \sigma(W_g[e; r] + b_g), \\ \text{Merge}(e, s) &= e + \alpha g \odot W_s r \end{aligned}$$

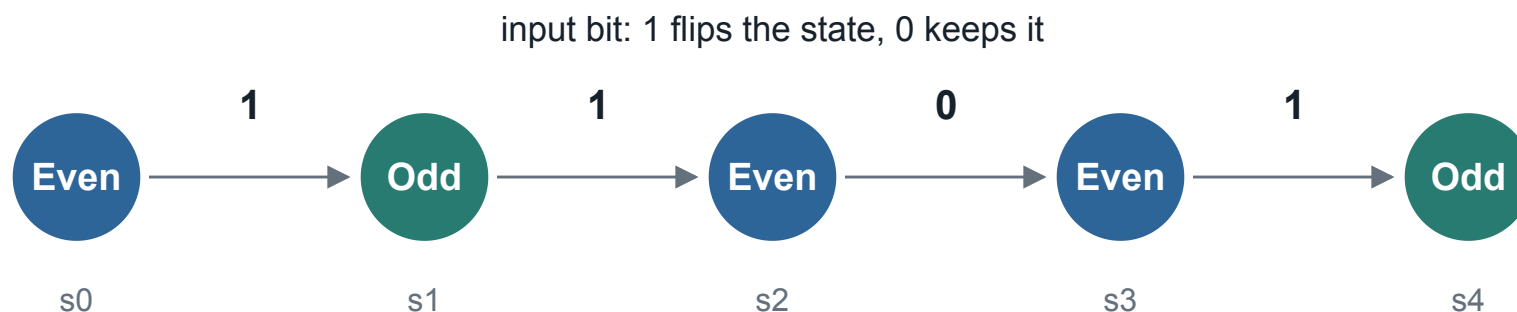
$$H_t = \mathcal{F}_{\Theta}(H_{t-1}; e_t, M_{\leq t}, t), \quad H_0 = (s_{\star}, \emptyset)$$

$$\mathcal{G}_{\Theta}(H) = \text{softmax}(W_o \text{Norm}_o(s)), \quad p_{\Theta}(\cdot \mid x_{1:t}) = \mathcal{G}_{\Theta}(H_t)$$

One transition updates both s_t and C_t^D , conditioned on encoder features and memory.

Feedback extends the computation across tokens

$$H_T = F_T \circ \dots \circ F_1(H_0), \quad F_t(H) = \mathcal{F}_\Theta(H; e_t, M_{\leq t}, t)$$



Per token

$L_E + L_D$ Transformer blocks.

The merge connects successive updates.

Known-token encoding is parallel. RLT-1 decoder updates follow token order.

Across T tokens

The feedback path traverses TL_D decoder blocks with parameters reused.

Length generalization depends on the depth split

Eight-layer models. Test accuracy at length 256, mean $\pm$ sample SD over three seeds.

Task	RLT-1 allocation	GPT Layer 8	RLT-1
Parity Train: 3–40 bits	Layer 5 + 3 or Layer 7 + 1	$50.07 \pm 1.63\%$	$100.00 \pm 0.00\%$
Swaps-S_5 Train: 32 operations	Layer 4 + 4	$0.85 \pm 0.30\%$	$97.30 \pm 2.76\%$

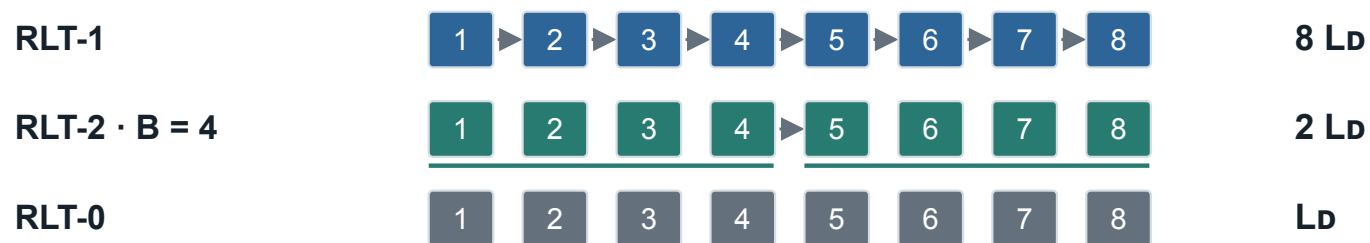
Parity favors a shallow decoder. Swap tracking benefits from more decoder layers.

2,000 training steps, seeds 42–44, 1,024 shared test examples per length.

Each run selects its checkpoint using minimum in-distribution validation loss.

Layer counts match. Parameters and compute differ: RLT-1 has 26.10–28.73M parameters, versus GPT's 25.31M.

Feedback frequency and parallelism



T = 8 known tokens; schematic

At Layer 4 + 4	Feedback interval	Parity / 64 bits	Swaps - S_5 / 64 ops
RLT-1	1 token	100.00 ± 0.00%	100.00 ± 0.00%
RLT-2	4 tokens	98.99 ± 1.66%	19.60 ± 6.54%

RLT-2 shares the previous boundary state within each chunk.

Known positions in a chunk run in parallel, then its last output updates feedback.

Final-answer/state accuracy, mean ± sample SD, seeds 42–44. Same checkpoint protocol as the previous slide.

State size and parallel evaluation

Model core	Update in the previous state	Persistent memory*	Known-token evaluation
Linear attention	Additive / affine	Fixed matrix and normalizer	Prefix sums or chunked contraction
Selective SSM	Input-conditioned affine	Fixed state	Structured scan or SSD
DeltaNet	Affine rank-one edit	Fixed matrix	Structured chunk algorithm
RLT-1	Nonlinear decoder cell	Bounded decoder state, growing encoder memory	Parallel encoder, sequential decoder

*At fixed width, depth, and local window. State sizes refer to inference, excluding training activations.

Note / Linear attention, SSD, and Delta Networks / RLT / State and execution

The transition determines the computation

Structured recurrent memory

$$h_t = A_t h_{t-1} + c_t$$

Linear attention, SSMs, and DeltaNet compress history through structured affine updates.

Nonlinear recurrent computation

$$H_t = \mathcal{F}_{\Theta}(H_{t-1}; e_t, M_{\leq t}, t)$$

RLT reuses a complete decoder cell across tokens and retains explicit context memory.

The state-space view connects the models.

The transition's structure determines what can be composed efficiently.

Open question: can a structured transition retain RLT's state-tracking gains?